



Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря
Сікорського» Факультет інформатики та обчислювальної
техніки
Кафедра інформатики і програмної інженерії

Звіт
з дисципліни «Аналіз даних в інформаційних системах»
Лабораторні роботи №1-2
"Створення процедур завантаження даних"

Виконав:

Студент II курсу
гр. ІП-33
Соколов О. В.

Перевірів:

Олійник Ю. О.

Лабораторні роботи № 1-2.

Створення процедур завантаження даних

Мета

Ознайомитись з підходами до створення сховищ даних; знайти дані для курсової роботи. ER-моделі предметної області (діаграм «Сутність-Зв'язок»)

Постановка задачі

При виконанні лабораторної роботи необхідно виконати наступні дії:

- 1) Самостійно обрати не менше 3-х джерел/файлів відкритих даних. У цілому в джерелах/файлах мають бути спільні 2+ виміри (наприклад, країни, континенти та роки) та сховище повинно містити не менше 5 вимірів;
- 2) Спроектувати модель сховища даних та Stage зони для ETL процесів;
- 3) Створити ETL засоби:
 - а. завантажити дані до Stage зони;
 - б. створити набір процедур/функцій для перетворення та завантаження даних до основного сховища (або створити засобами програмних ETL засобів);
 - с. передбачити можливість завантаження змінених та додаткових даних.

Виконання

Для виконання лабораторної роботи було обрано декілька наборів даних з сайту <https://www.kaggle.com/>, які пов'язані з індустрією кіно-фільмів.

Опис джерел даних

Файл movies.csv містить інформацію про 6820 фільмів випущених в 1986-2016 роках, 220 фільмів на кожен рік. Джерело: <https://www.kaggle.com/danielgrijalvas/movies>.

Таблиця 1.1 – Структура movies.csv

Назва поля	Призначення
name	Name of the movie
budget	The budget of a movie
company	The production company
country	Country of origin
director	The director
genre	Main genre of the movie.
gross	Revenue of the movie
rating	Rating of the movie (R, PG, etc.)
released	Release date (YYYY-MM-DD)
runtime	Duration of the movie
score	Imdb user rating
votes	Number of user votes
star	Main actor/actress
writer	Writer of the movie
year	Year of release

Файл the_oscar_award.csv містить інформацію про результати конкурсу «Оскар» з 1927 по 2025 роки. Джерело: <https://www.kaggle.com/datasets/unanimad/the-oscar-award>.

Таблиця 1.2 – Структура the_oscar_award.csv

Назва поля	Призначення
year_film	Film screened at
year_ceremony	Ceremony happened at
ceremony	Number of ceremony
category	Name of the category (as per official nomination)
canon_category	Name of the category, consistent across years
name	Slash separated names of the nominees
film	Film's title
winner	True if winner

Файли “2000-2009 Movies Box Office Collection.csv”, “2010-2024 Movies Box Office Collection.csv” і “2024 Movies Box Office Collection.csv” містять інформацію про касові збори фільмів з 2000 по 2024 роки відповідно. Джерело: <https://www.kaggle.com/datasets/parthdande/movies-box-office-collection-data-2000-2024>.

Таблиця 1.3 – Структура файлів “Movies Box Office Collection”

Назва поля	Призначення
Rank	Rank of a film
Release Group	Name of a film
Worldwide	Worldwide revenue
Domestic	The total earnings (in USD) from the domestic market (e.g., United States)
Domestic_percent	The percentage of the movie's total revenue that was earned domestically.
Foreign	The total earnings (in USD) from international markets (outside the domestic market)
Foreign_percent	The percentage of the movie's total revenue that was earned from foreign markets.
year	The release year of the movie.

Файл «movie_statistic_dataset.csv» містить загальну інформацію про фільм, режисера, бюджет. Джерело: <https://www.kaggle.com/datasets/alessandrolobello/the-ultimate-film-statistics-dataset-for-ml>.

Таблиця 1.4 – Структура файлу “movie_statistic_dataset.csv”

Назва поля	Призначення
Movie_title	Movie name
Production_date	Date of production
Genres	Genres
Runtime_minutes	Runtime in minutes
Director_name (primaryName)	Name of the director
Director_professions (primaryProfession)	Director professions
Director_birthYear	Director year of birth
Director_deathYear	Director year of death
Movie_averageRating	Average rating
Movie_numberOfVotes	Number of votes
Approval_Index	Is a normalized indicator (on scale 0-10) calculated by multiplying the logarithm of the number of
Production_budget	Production budget in \$
Domestic_gross	Domestic gross in \$
Worldwide_gross	Worldwide gross in \$

Підготовка даних

Для зручної подальшої роботи з даними, було створено нові csv файли на основі обраних наборів даних. Зміни, які були зроблені:

Очищення колонок в наборах даних:

1. Box Office:
 - a. Видалення колонки "Rank".
 - b. Очищення колонок Worldwide, Domestic, Foreign: видалення ком, перетворення значень на числа.
 - c. Очищення колонок Domestic_percent та Foreign_percent: видалення знаків %, заміна <0.1 на 0, перетворення значень на десяткові числа (ділення на 100).

- d. Перетворення значення року (year) в ціле число.
 - e. Перейменування колонок для узгодження з базою даних.
2. Movies:
- a. Очищення колонок score та votes (перетворення на числові значення, заміна помилок на NaN).
 - b. Очищення колонок budget та gross (перетворення на числові значення).
 - c. Очищення колонки runtime (перетворення на числове значення).
 - d. Обробка дат випуску (вилучення формату дати і перетворення в datetime).
 - e. Видалення рядків без дат випуску.
 - f. Перетворення року на ціле число.
3. Oscar Awards:
- a. Видалення рядків з відсутніми значеннями.
 - b. Перетворення колонок year_film, year_ceremony та ceremony в цілі числа.
 - c. Перетворення колонки winner в булевий тип.
4. Movie Budget:
- a. Перейменування колонок для узгодження з базою даних.
 - b. Очищення колонок runtime_minutes, movie_average_rating, movie_number_of_votes, approval_index, production_budget, domestic_gross, worldwide_gross (перетворення на числові значення, заміна помилок на NaN).
 - c. Очищення дати виробництва (production_date), перетворення на datetime.
 - d. Очищення колонок director_birth_year та director_death_year, заміна недійсних значень на NaN.
 - e. Перетворення колонок director_birth_year та director_death_year на цілі числа.
 - f. Форматування колонок director_professions та genres у відповідний формат для бази даних (масиви).
 - g. Заміна недійсних значень на NaN.

Код скрипту:

```
import pandas as pd
import os
```

```
os.makedirs("da_lab1/clean_data", exist_ok=True)
```

```

def clean_box_office_data(file_path, output_path):
    """Clean specific columns in the box office dataset."""
    print(f"Processing {file_path}...")

    df = pd.read_csv(file_path)

    df = df.drop(columns=["Rank"])

    df["Worldwide"] = df["Worldwide"].replace({" ": ""},
regex=True).astype(float)
    df["Domestic"] = df["Domestic"].replace({" ": ""},
regex=True).astype(float)
    df["Foreign"] = df["Foreign"].replace({" ": ""},
regex=True).astype(float)

    df["Domestic_percent"] = (
        df["Domestic_percent"]
        .replace({" ": "", "%": "", "<0.1": "0"}, regex=True)
        .astype(float)
        / 100
    )
    df["Foreign_percent"] = (
        df["Foreign_percent"]
        .replace({" ": "", "%": "", "<0.1": "0"}, regex=True)
        .astype(float)
        / 100
    )

    df["year"] = df["year"].astype(int)

    df = df.rename(
        columns={
            "Release Group": "release_group",
            "Worldwide": "worldwide",
            "Domestic": "domestic",
            "Domestic_percent": "domestic_percent",
            "Foreign": "foreign",
            "Foreign_percent": "foreign_percent",
            "year": "year",
        }
    )

    df.to_csv(output_path, index=False)
    print(f"✅ Cleaned data saved to {output_path}")

```

```

def clean_movie_data(file_path, output_path):
    """Clean specific columns in the movie dataset."""
    print(f"Processing {file_path}...")

    df = pd.read_csv(file_path)

    df["score"] = pd.to_numeric(df["score"], errors="coerce")

    df["votes"] = (
        df["votes"]
        .apply(lambda x: int(float(x)) if pd.notnull(x) else None)
        .astype(pd.Int64Dtype())
    )

    df["budget"] = pd.to_numeric(df["budget"], errors="coerce")
    df["gross"] = pd.to_numeric(df["gross"], errors="coerce")

    df["runtime"] = pd.to_numeric(df["runtime"], errors="coerce")

    df["released"] = df["released"].str.extract(r"([a-zA-Z]+)\s\d{1,2},\s\d{4}")
    df["released"] = pd.to_datetime(df["released"],
    errors="coerce")

    df = df.dropna(subset=["released"])

    df["year"] = df["year"].astype(int)

    df.to_csv(output_path, index=False)
    print(f"✅ Cleaned data saved to {output_path}")

```

```

def clean_oscar_award_data(file_path, output_path):
    """Clean specific columns in the oscar award dataset."""
    print(f"Processing {file_path}...")

    df = pd.read_csv(file_path)

    df = df.dropna()

    df["year_film"] = df["year_film"].astype(int)
    df["year_ceremony"] = df["year_ceremony"].astype(int)

```



```

df["ceremony"] = df["ceremony"].astype(int)

df["winner"] = df["winner"].astype(bool)

df.to_csv(output_path, index=False)
print(f"✅ Cleaned data saved to {output_path}")

def clean_movie_budget_data(file_path, output_path):
    """Clean specific columns in the movie budget dataset."""
    print(f"Processing {file_path}...")

    df = pd.read_csv(file_path)

    df = df.rename(
        columns={
            "movie_title": "movie_title",
            "production_date": "production_date",
            "genres": "genres",
            "runtime_minutes": "runtime_minutes",
            "director_name": "director_name",
            "director_professions": "director_professions",
            "director_birthYear": "director_birth_year",
            "director_deathYear": "director_death_year",
            "movie_averageRating": "movie_average_rating",
            "movie_numOfVotes": "movie_number_of_votes",
            "approval_Index": "approval_index",
            "Production budget $": "production_budget",
            "Domestic gross $": "domestic_gross",
            "Worldwide gross $": "worldwide_gross",
        }
    )

    df["runtime_minutes"] = pd.to_numeric(df["runtime_minutes"],
errors="coerce")
    df["movie_average_rating"] = pd.to_numeric(
        df["movie_average_rating"], errors="coerce"
    )
    df["movie_number_of_votes"] = pd.to_numeric(
        df["movie_number_of_votes"], errors="coerce"
    ).astype("Int64")
    df["approval_index"] = pd.to_numeric(df["approval_index"],
errors="coerce")

```

```

df["production_budget"] =
pd.to_numeric(df["production_budget"], errors="coerce")
df["domestic_gross"] = pd.to_numeric(df["domestic_gross"],
errors="coerce")
df["worldwide_gross"] = pd.to_numeric(df["worldwide_gross"],
errors="coerce")

df["production_date"] = pd.to_datetime(df["production_date"],
errors="coerce")

df["director_birth_year"] = df["director_birth_year"].replace(
    ["-", "\\N", "alive"], pd.NA
)
df["director_death_year"] = df["director_death_year"].replace(
    ["-", "\\N", "alive"], pd.NA
)

df["director_birth_year"] =
df["director_birth_year"].astype(pd.Int64Dtype())
df["director_death_year"] =
df["director_death_year"].astype(pd.Int64Dtype())

df["director_professions"] = df["director_professions"].apply(
    lambda x: "{" + x.replace(",", " ") + "}" if pd.notnull(x)
else None
)

df["genres"] = df["genres"].apply(
    lambda x: "{" + ",".join(x.split(",")) + "}" if
pd.notnull(x) else None
)

df.replace({"-": pd.NA, "{-}": pd.NA}, inplace=True)

df.to_csv(output_path, index=False)
print(f"✅ Cleaned data saved to {output_path}")

clean_box_office_data(
    "da_lab1/data/box_office.csv",
    "da_lab1/clean_data/box_office.csv"
)

```

```
clean_movie_data("da_lab1/data/movies.csv",  
"da_lab1/clean_data/movies.csv")
```

```
clean_oscar_award_data(  
    "da_lab1/data/oscar_award.csv",  
    "da_lab1/clean_data/oscar_award.csv"  
)
```

```
clean_movie_budget_data(  
    "da_lab1/data/movie_budget.csv",  
    "da_lab1/clean_data/movie_budget.csv"  
)
```

```
print("✅ Data cleaning complete! Cleaned files are in  
'clean_data/'")
```

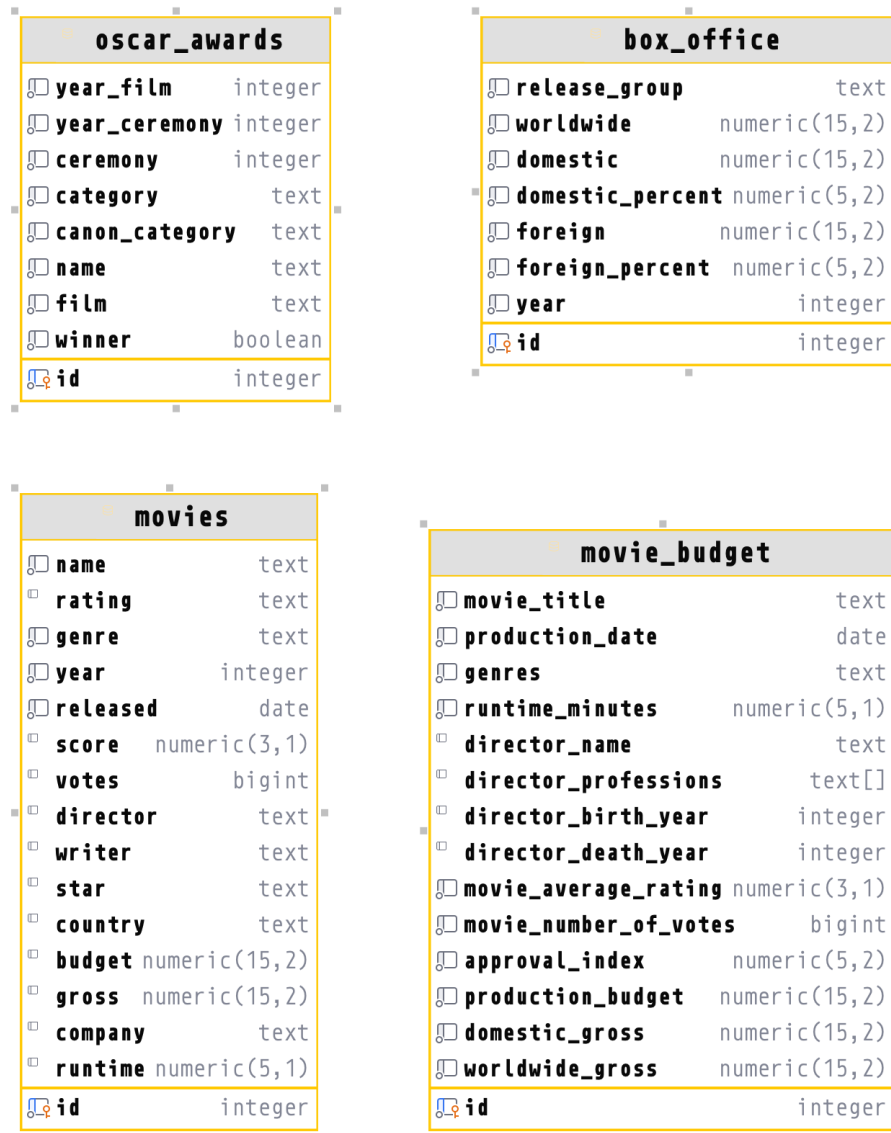



Рис. 2.1 – Діаграма сутностей stage зони

Таблиця **box_office** містить:

1. **id** — це автоматично збільшуване ціле число (SERIAL), яке використовується як первинний ключ для ідентифікації кожного запису.
2. **release_group** — текстове поле (TEXT), яке зберігає назву групи фільмів або франшизи, до якої належить фільм.
3. **worldwide** — числове поле (NUMERIC(15, 2)), яке містить загальні світові збори фільму в доларах США.
4. **domestic** — числове поле (NUMERIC(15, 2)), яке містить суму зборів фільму в домашньому (вітчизняному) ринку (наприклад, США).
5. **domestic_percent** — числове поле (NUMERIC(5, 2)), яке відображає відсоток зборів фільму, отриманих на вітчизняному ринку.

6. `foreign` — числове поле (`NUMERIC(15, 2)`), яке містить збори фільму на міжнародному ринку.
7. `foreign_percent` — числове поле (`NUMERIC(5, 2)`), яке відображає відсоток зборів фільму, отриманих на міжнародному ринку.
8. `year` — ціле число (`INTEGER`), яке містить рік випуску фільму.

Таблиця `movie_budget` містить:

1. `id` — це автоматично збільшуване ціле число (`SERIAL`), яке використовується як первинний ключ для ідентифікації кожного запису.
2. `movie_title` — текстове поле (`TEXT`), яке містить назву фільму.
3. `production_date` — дата (`DATE`), яка вказує на дату виробництва фільму.
4. `genres` — текстове поле (`TEXT`), яке містить жанри фільму, розділені комами.
5. `runtime_minutes` — числове поле (`NUMERIC(5, 1)`), яке вказує тривалість фільму в хвилинах.
6. `director_name` — текстове поле (`TEXT`), яке зберігає ім'я режисера фільму.
7. `director_professions` — масив текстових значень (`TEXT[]`), який містить професії режисера (наприклад, режисер, продюсер тощо).
8. `director_birth_year` — ціле число (`INTEGER`), яке вказує рік народження режисера.
9. `director_death_year` — ціле число (`INTEGER`), яке вказує рік смерті режисера (якщо застосовно).
10. `movie_average_rating` — числове поле (`NUMERIC(3, 1)`), яке містить середній рейтинг фільму.
11. `movie_number_of_votes` — велике ціле число (`BIGINT`), яке містить кількість голосів за фільм.
12. `approval_index` — числове поле (`NUMERIC(5, 2)`), яке є індексом схвалення фільму, розрахованим як логарифм кількості голосів, помножений на середній рейтинг.
13. `production_budget` — числове поле (`NUMERIC(15, 2)`), яке містить бюджет виробництва фільму в доларах.

14. `domestic_gross` — числове поле (`NUMERIC(15, 2)`), яке містить збори фільму на вітчизняному ринку в доларах.

15. `worldwide_gross` — числове поле (`NUMERIC(15, 2)`), яке містить збори фільму на світовому рівні в доларах.

Таблиця `movies` містить:

1. `id` — це автоматично збільшуване ціле число (`SERIAL`), яке використовується як первинний ключ для ідентифікації кожного запису.

2. `name` — текстове поле (`TEXT`), яке містить назву фільму.

3. `rating` — текстове поле (`TEXT`), яке вказує рейтинг фільму (наприклад, R, PG тощо).

4. `genre` — текстове поле (`TEXT`), яке містить основний жанр фільму.

5. `year` — ціле число (`INTEGER`), яке містить рік випуску фільму.

6. `released` — дата (`DATE`), яка вказує на дату виходу фільму.

7. `score` — числове поле (`NUMERIC(3, 1)`), яке вказує оцінку фільму на IMDb.

8. `votes` — велике ціле число (`BIGINT`), яке містить кількість голосів на IMDb.

9. `director` — текстове поле (`TEXT`), яке містить ім'я режисера фільму.

10. `writer` — текстове поле (`TEXT`), яке містить ім'я сценариста фільму.

11. `star` — текстове поле (`TEXT`), яке містить ім'я головного актора чи актриси.

12. `country` — текстове поле (`TEXT`), яке містить країну виробництва фільму.

13. `budget` — числове поле (`NUMERIC(15, 2)`), яке містить бюджет фільму (може бути нульовим, якщо не вказано).

14. `gross` — числове поле (`NUMERIC(15, 2)`), яке містить загальні збори фільму.

15. `company` — текстове поле (`TEXT`), яке містить назву компанії, яка виробила фільм.

16. runtime — числове поле (NUMERIC(5, 1)), яке містить тривалість фільму в хвилинах.

Таблиця oscar_awards містить:

- id — це автоматично збільшуване ціле число (SERIAL), яке використовується як первинний ключ для ідентифікації кожного запису.
- year_film — ціле число (INTEGER), яке вказує рік, у якому фільм був випущений.
- year_ceremony — ціле число (INTEGER), яке вказує рік церемонії вручення Оскарів.
- ceremony — ціле число (INTEGER), яке вказує на номер церемонії.
- category — текстове поле (TEXT), яке містить назву категорії нагороди.
- canon_category — текстове поле (TEXT), яке містить стандартну назву категорії, яка використовується на всіх церемоніях.
- name — текстове поле (TEXT), яке містить імена номінантів, розділені слешем.
- film — текстове поле (TEXT), яке містить назву фільму, який був номінований.
- winner — булеве значення (BOOLEAN), яке вказує, чи виграв фільм в цій категорії.

Код створення stage зони:

```
CREATE SCHEMA IF NOT EXISTS staging;
```

```
CREATE TABLE staging.box_office
(
    id                SERIAL PRIMARY KEY,
    release_group     TEXT                NOT NULL,
    worldwide         NUMERIC(15, 2) NOT NULL,
    domestic          NUMERIC(15, 2) NOT NULL,
    domestic_percent  NUMERIC(5, 2)  NOT NULL,
    "foreign"         NUMERIC(15, 2) NOT NULL,
    foreign_percent   NUMERIC(5, 2)  NOT NULL,
    year              INTEGER           NOT NULL
);
```

```
CREATE TABLE staging.movie_budget
(
    id                SERIAL PRIMARY KEY,
    movie_title       TEXT                NOT NULL,
    production_date   DATE                NOT NULL,
    genres            TEXT                NOT NULL,
    runtime_minutes   NUMERIC(5, 1) NOT NULL,
    director_name     TEXT                NULL,
    director_professions TEXT[],
    director_birth_year INTEGER           NULL,
    director_death_year INTEGER           NULL,
    movie_average_rating NUMERIC(3, 1) NOT NULL,
    movie_number_of_votes BIGINT          NOT NULL,
    approval_index     NUMERIC(5, 2) NOT NULL,
    production_budget  NUMERIC(15, 2) NOT NULL,
    domestic_gross     NUMERIC(15, 2) NOT NULL,
    worldwide_gross    NUMERIC(15, 2) NOT NULL
);
```

```
CREATE TABLE staging.movies
(
    id                SERIAL PRIMARY KEY,
    name              TEXT                NOT NULL,
    rating            TEXT                NULL,
    genre             TEXT                NOT NULL,
    year              INTEGER           NOT NULL,
    released          DATE                NOT NULL,
    score             NUMERIC(3, 1) NULL,
```

```

votes      BIGINT          NULL,
director   TEXT            NULL,
writer     TEXT            NULL,
star       TEXT            NULL,
country    TEXT            NULL,
budget     NUMERIC(15, 2)  NULL,
gross      NUMERIC(15, 2)  NULL,
company    TEXT            NULL,
runtime    NUMERIC(5, 1)   NULL
);

CREATE TABLE staging.oscar_awards
(
    id          SERIAL PRIMARY KEY,
    year_film   INTEGER NOT NULL,
    year_ceremony INTEGER NOT NULL,
    ceremony    INTEGER NOT NULL,
    category    TEXT     NOT NULL,
    canon_category TEXT   NOT NULL,
    name        TEXT     NOT NULL,
    film        TEXT     NOT NULL,
    winner      BOOLEAN NOT NULL
);

```

Заповнення stage зони

```
COPY staging.box_office (release_group, worldwide, domestic,  
domestic_percent, "foreign", foreign_percent, year)  
FROM '/private/tmp/box_office.csv' DELIMITER ',' CSV HEADER;
```

```
COPY staging.movies (name, rating, genre, year, released, score,  
votes, director, writer, star, country, budget, gross, company,  
runtime)  
FROM '/private/tmp/movies.csv' DELIMITER ',' CSV HEADER;
```

```
COPY staging.oscar_awards(year_film, year_ceremony, ceremony,  
category, canon_category, name, film, winner)  
FROM '/private/tmp/oscar_award.csv' DELIMITER ',' CSV HEADER;
```

```
COPY staging.movie_budget (movie_title, production_date, genres,  
runtime_minutes, director_name, director_professions,  
director_birth_year, director_death_year, movie_average_rating,  
movie_number_of_votes, approval_index, production_budget,  
domestic_gross, worldwide_gross)  
FROM '/private/tmp/movie_budget.csv' DELIMITER ',' CSV HEADER;
```

	1	2	3	4	5	6	7	8
id	1	2	3	4	5	6	7	8
release_group	Mission: Impossible II	Gladiator	Cast Away	What Women Want	Dinosaur	How the Grinch Stole Christmas	Meet the Parents	The Perfectionist
worldwide	546388108.00	460583960.00	429632142.00	374111707.00	349822765.00	345842198.00	330444045.00	328718430.00
domestic	215409889.00	187705427.00	233632142.00	182811707.00	137748063.00	260745620.00	166244045.00	182618430.00
domestic_percent	0.39	0.41	0.54	0.49	0.39	0.75	0.50	0.56
foreign	330978219.00	272878533.00	196000000.00	191300000.00	212074702.00	85096578.00	164200000.00	146100000.00
foreign_percent	0.61	0.59	0.46	0.51	0.61	0.25	0.50	0.44
year	2000	2000	2000	2000	2000	2000	2000	2000

Рис. 3.1 – Таблиця stage зони box_office

	1	2	3	4	5
id	1	2	3	4	5
movie_title	Avatar: The Way of Water	Avengers: Endgame	Pirates of the Caribbean: On Stranger Tides	Avengers: Age of Ultron	Avengers
production_date	2022-12-09	2019-04-23	2011-05-20	2015-04-22	2018-04-
genres	{Action,Adventure,Fantasy}	{Action,Adventure,Drama}	{Action,Adventure,Fantasy}	{Action,Adventure,Sci-Fi}	{Action,
runtime_minutes	192.0	181.0	137.0	141.0	149.0
director_name	James Cameron	<null>	Rob Marshall	Joss Whedon	<null>
director_professions	{writer,producer,director}	<null>	{director,miscellaneous,producer}	{writer,producer,director}	<null>
director_birth_year	1954	<null>	1968	1964	<null>
director_death_year	<null>	<null>	<null>	<null>	<null>
movie_average_rating	7.8	8.4	6.6	7.3	8.4
movie_number_of_votes	277543	1143642	533763	878573	1091968
approval_index	7.06	8.49	6.27	7.21	8.46
production_budget	460000000.00	400000000.00	379000000.00	365000000.00	300000000.00
domestic_gross	667830256.00	858373000.00	241071802.00	459005868.00	67881548.00
worldwide_gross	2265935552.00	2794731755.00	1045713802.00	1395316979.00	20483597.00

Рис. 3.2 – Таблица stage зоны movie_budget

	1	2	3	4	5	6	7
id	80	81	82	83	84	85	86
name	The Shining	The Blue Lagoon	Star Wars: Episode V – The Empire Strikes Back	Airplane!	Caddyshack	Friday the 13th	The E
rating	R	R	PG	PG	R	R	R
genre	Drama	Adventure	Action	Comedy	Comedy	Horror	Action
year	1980	1980	1980	1980	1980	1980	1980
released	1980-06-13	1980-07-02	1980-06-20	1980-07-02	1980-07-25	1980-05-09	1980-
score	8.4	5.8	8.7	7.7	7.3	6.4	7.9
votes	927000	65000	1200000	221000	108000	123000	188000
director	Stanley Kubrick	Randal Kleiser	Irvin Kershner	Jim Abrahams	Harold Ramis	Sean S. Cunningham	John
writer	Stephen King	Henry De Vere Stacpoole	Leigh Brackett	Jim Abrahams	Brian Doyle-Murray	Victor Miller	Dan A
star	Jack Nicholson	Brooke Shields	Mark Hamill	Robert Hays	Chevy Chase	Betsy Palmer	John
country	United Kingdom	United States	United States	United Stat.	United States	United States	United
budget	19000000.00	4500000.00	18000000.00	3500000.00	6000000.00	550000.00	27000
gross	46998772.00	58853106.00	538375067.00	83453539.00	39846344.00	39754601.00	11522
company	Warner Bros.	Columbia Pictures	Lucasfilm	Paramount P.	Orion Pictures	Paramount Pictures	Unive
runtime	146.0	104.0	124.0	88.0	98.0	95.0	133.0

Рис. 3.3 – Таблица stage зоны movies

	1	2	3	4	5	6
id	1	2	3	4	5	6
year_film	1927	1927	1927	1927	1927	1927
year_ceremony	1928	1928	1928	1928	1928	1928
ceremony	1	1	1	1	1	1
category	ACTOR	ACTOR	ACTOR	ACTOR	ACTRESS	ACTRESS
canon_category	ACTOR IN A LEADING ROLE	ACTOR IN A LEADING ROLE	ACTOR IN A LEADING ROLE	ACTOR IN A LEADING ROLE	ACTRESS IN A LEADING ROLE	ACTRESS IN A LEADING ROLE
name	Richard Barthelmess	Richard Barthelmess	Emil Jannings	Emil Jannings	Louise Dresser	Janet Gaynor
film	The Noose	The Patent Leather Kid	The Last Command	The Way of All Flesh	A Ship Comes In	7th Heaven
winner	false	false	true	true	false	true

Рис. 3.4 – Таблиця stage зони oscar_awards

Дані було успішно завантажено до stage зони.

Модель основного сховища

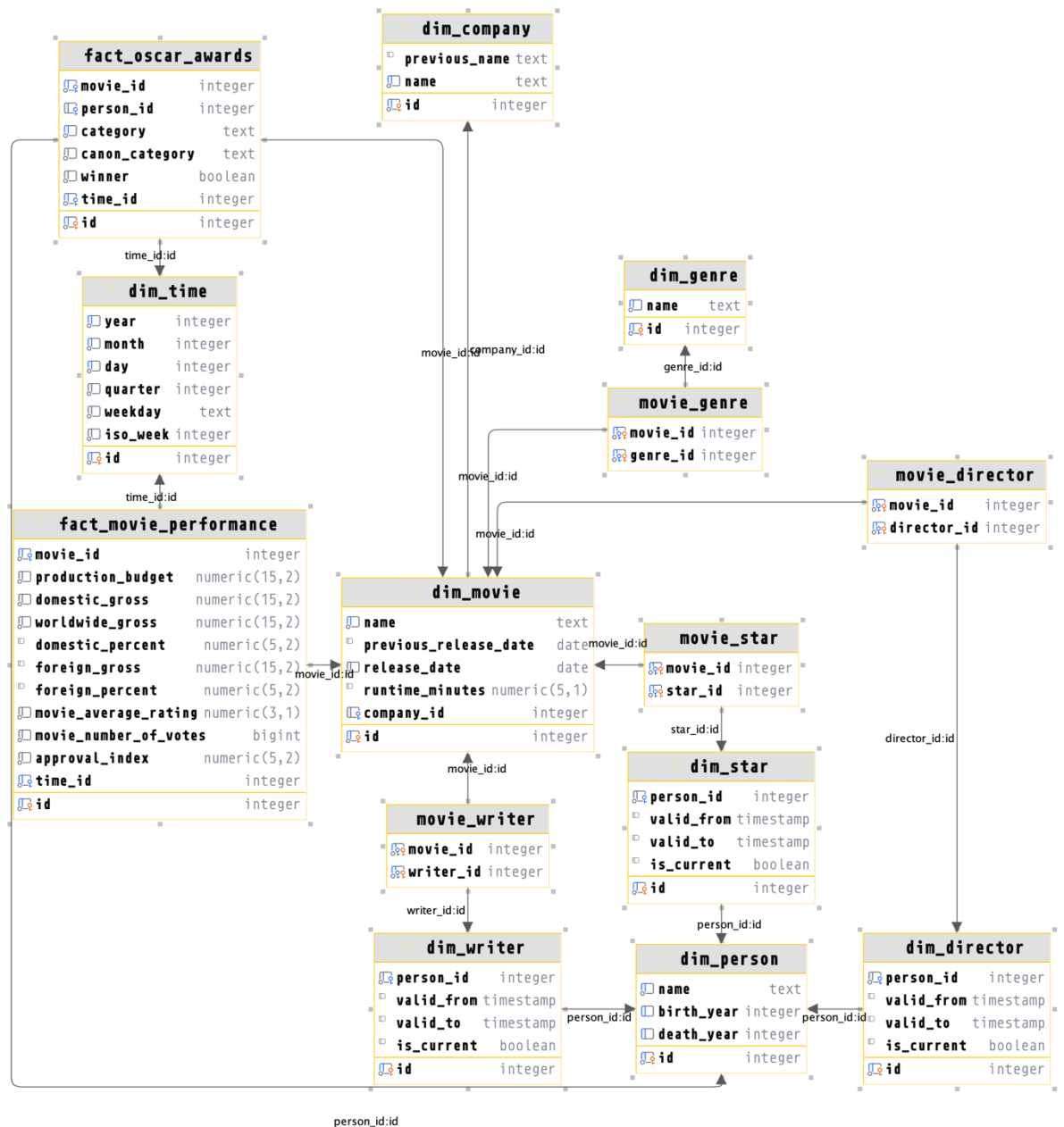


Рис. 4.1 – Діаграма сутностей main зони

Таблиця **dim_company**: Це таблиця виміру, яка містить інформацію про кінокомпанії. Вона дозволяє відстежувати зміни в назвах компаній завдяки використанню Slowly Changing Dimension Type 3 (SCD Type 3). Це корисно для аналізу історичних змін у брендингу та володінні студіями.

Таблиця **dim_person**: Це таблиця виміру, яка містить інформацію про осіб, що беруть участь у кіновиробництві. Вона включає імена, роки

народження та смерті, що дозволяє аналізувати кар'єру кінематографістів та їхній внесок у розвиток кіноіндустрії.

Таблиця **dim_genre**: Це таблиця виміру, яка містить список жанрів фільмів. Вона використовується для класифікації фільмів за жанровою приналежністю, що дозволяє здійснювати аналіз популярності жанрів та їх впливу на касові збори.

Таблиця **dim_time**: Це таблиця виміру, яка містить дані про дату в різних розрізах: рік, місяць, день, квартал, день тижня та номер тижня за стандартом ISO. Вона дозволяє проводити часовий аналіз касових зборів, випуску фільмів та інших кінематографічних подій.

Таблиця **dim_movie**: Це таблиця виміру, яка містить основну інформацію про фільми. Вона використовує Slowly Changing Dimension Type 3 (SCD Type 3) для відстеження змін дати релізу. Додатково вона включає тривалість фільму та пов'язану кінокомпанію, що дозволяє детальніше аналізувати історію виробництва фільмів.

Таблиця **dim_director**: Це таблиця виміру, яка містить дані про режисерів. Вона використовує Slowly Changing Dimension Type 2 (SCD Type 2), що дозволяє відстежувати зміни у зв'язку режисера з фільмами в часі, зберігаючи історичні записи.

Таблиця **dim_writer**: Це таблиця виміру, яка містить інформацію про сценаристів. Вона реалізує Slowly Changing Dimension Type 2 (SCD Type 2), що дозволяє аналізувати зміни у кар'єрі сценаристів та їхній вплив на кіновиробництво.

Таблиця **dim_star**: Це таблиця виміру, яка містить інформацію про акторів. Використання Slowly Changing Dimension Type 2 (SCD Type 2) дозволяє фіксувати зміни у їхній кар'єрі та ролях у фільмах у різні періоди часу.

Таблиця **fact_movie_performance**: Це фактова таблиця, яка містить дані про фінансові показники фільмів. Вона включає бюджет, касові збори

на внутрішньому та міжнародному ринках, відсоткове співвідношення зборів, середній рейтинг та кількість голосів. Це основне джерело даних для аналізу фінансового успіху фільмів у поєднанні з часовими даними.

Таблиця **fact_oscar_awards**: Це фактова таблиця, яка містить інформацію про нагороди Оскара. Вона включає категорії номінацій, переможців, номінантів та пов'язану особу (режисер, сценарист, актор). Використання цієї таблиці дозволяє аналізувати динаміку перемог та номінацій фільмів у різні роки.

Таблиця **movie_genre**: Це таблиця зв'язку, яка дозволяє встановлювати зв'язки між фільмами та жанрами. Вона реалізує багато-до-багатьох зв'язок, що дає можливість аналізувати популярність жанрів у різні періоди часу.

Таблиця **movie_director**: Це таблиця зв'язку, яка містить відношення між фільмами та їх режисерами. Вона дозволяє визначати вплив конкретних режисерів на успіх фільмів та аналізувати їхній внесок у кіноіндустрію.

Таблиця **movie_writer**: Це таблиця зв'язку, яка встановлює відношення між фільмами та сценаристами. Вона дозволяє проводити дослідження щодо впливу сценаристів на успіх фільмів та взаємозв'язку між різними роботами одного автора.

Таблиця **movie_star**: Це таблиця зв'язку, яка містить зв'язки між фільмами та акторами. Вона використовується для аналізу популярності акторів, їхньої участі у різних фільмах та впливу на касові збори фільмів.

Код створення main зони:

```
CREATE SCHEMA IF NOT EXISTS main;  
  
-- Dimension Tables  
  
-- SCD Type 3  
CREATE TABLE main.dim_company
```



```
(
    id      SERIAL PRIMARY KEY,
    previous_name TEXT NULL,
    name TEXT NOT NULL UNIQUE
);
```

```
CREATE TABLE main.dim_person
(
    id          SERIAL PRIMARY KEY,
    name        TEXT NOT NULL UNIQUE,
    birth_year  INTEGER,
    death_year  INTEGER,
    CONSTRAINT unique_person UNIQUE (name, birth_year, death_year)
);
```

```
CREATE TABLE main.dim_genre
(
    id      SERIAL PRIMARY KEY,
    name TEXT NOT NULL UNIQUE
);
```

```
CREATE TABLE main.dim_time
(
    id          SERIAL PRIMARY KEY,
    year        INTEGER NOT NULL,
    month       INTEGER NOT NULL,
    day         INTEGER NOT NULL,
    quarter     INTEGER NOT NULL,
    weekday     TEXT      NOT NULL,
    iso_week    INTEGER NOT NULL,
    CONSTRAINT unique_date UNIQUE (year, month, day)
);
```

-- SCD Type 3

```
CREATE TABLE main.dim_movie
(
    id          SERIAL PRIMARY KEY,
    name        TEXT NOT NULL UNIQUE,
    previous_release_date DATE NULL,
    release_date DATE NOT NULL,
    runtime_minutes NUMERIC(5, 1),
    company_id  INT,
    CONSTRAINT fk_company FOREIGN KEY (company_id) REFERENCES
main.dim_company (id)
);
```

-- SCD Type 2

```
CREATE TABLE main.dim_director
(
    id          SERIAL PRIMARY KEY,
```

```

        person_id INT NOT NULL,
        valid_from TIMESTAMP DEFAULT now(),
        valid_to TIMESTAMP NULL,
        is_current BOOLEAN DEFAULT TRUE,
        CONSTRAINT fk_person FOREIGN KEY (person_id) REFERENCES
main.dim_person (id)
);

-- SCD Type 2
CREATE TABLE main.dim_writer
(
    id SERIAL PRIMARY KEY,
    person_id INT NOT NULL,
    valid_from TIMESTAMP DEFAULT now(),
    valid_to TIMESTAMP NULL,
    is_current BOOLEAN DEFAULT TRUE,
    CONSTRAINT fk_person FOREIGN KEY (person_id) REFERENCES
main.dim_person (id)
);

-- SCD Type 2
CREATE TABLE main.dim_star
(
    id SERIAL PRIMARY KEY,
    person_id INT NOT NULL,
    valid_from TIMESTAMP DEFAULT now(),
    valid_to TIMESTAMP NULL,
    is_current BOOLEAN DEFAULT TRUE,
    CONSTRAINT fk_person FOREIGN KEY (person_id) REFERENCES
main.dim_person (id)
);

-- Fact Table
CREATE TABLE main.fact_movie_performance
(
    id SERIAL PRIMARY KEY,
    movie_id INT NOT NULL,
    production_budget NUMERIC(15, 2) NOT NULL,
    domestic_gross NUMERIC(15, 2) NOT NULL,
    worldwide_gross NUMERIC(15, 2) NOT NULL,
    domestic_percent NUMERIC(5, 2) GENERATED ALWAYS AS
        (100 * domestic_gross / NULLIF(worldwide_gross, 0)) STORED,
    foreign_gross NUMERIC(15, 2) GENERATED ALWAYS AS
        (worldwide_gross - domestic_gross) STORED,
    foreign_percent NUMERIC(5, 2) GENERATED ALWAYS AS
        (100 * (worldwide_gross - domestic_gross) /
NULLIF(worldwide_gross, 0)) STORED,
    movie_average_rating NUMERIC(3, 1) NOT NULL,
    movie_number_of_votes BIGINT NOT NULL,
    approval_index NUMERIC(5, 2) NOT NULL,
    time_id INT NOT NULL,
    CONSTRAINT fk_movie FOREIGN KEY (movie_id) REFERENCES

```

```

main.dim_movie (id),
    CONSTRAINT fk_time FOREIGN KEY (time_id) REFERENCES
main.dim_time (id),
    CONSTRAINT unique_movie_time UNIQUE (movie_id, time_id)
);

```

```

CREATE TABLE main.fact_oscar_awards
(
    id                SERIAL PRIMARY KEY,
    movie_id          INT      NOT NULL,
    person_id         INT      NULL,
    category          TEXT     NOT NULL,
    canon_category    TEXT     NOT NULL,
    winner            BOOLEAN NOT NULL,
    time_id           INT      NOT NULL,
    CONSTRAINT fk_movie FOREIGN KEY (movie_id) REFERENCES
main.dim_movie (id),
    CONSTRAINT fk_person FOREIGN KEY (person_id) REFERENCES
main.dim_person (id),
    CONSTRAINT fk_time FOREIGN KEY (time_id) REFERENCES
main.dim_time (id),
    CONSTRAINT unique_movie_time_category UNIQUE (movie_id,
time_id, category)
);

```

-- Movie-Genre Relationship (Many-to-Many)

```

CREATE TABLE main.movie_genre
(
    movie_id INT NOT NULL,
    genre_id INT NOT NULL,
    PRIMARY KEY (movie_id, genre_id),
    CONSTRAINT fk_movie FOREIGN KEY (movie_id) REFERENCES
main.dim_movie (id),
    CONSTRAINT fk_genre FOREIGN KEY (genre_id) REFERENCES
main.dim_genre (id)
);

```

-- Movie-Director Relationship (Many-to-Many)

```

CREATE TABLE main.movie_director
(
    movie_id    INT NOT NULL,
    director_id INT NOT NULL,
    PRIMARY KEY (movie_id, director_id),
    CONSTRAINT fk_movie FOREIGN KEY (movie_id) REFERENCES
main.dim_movie (id),
    CONSTRAINT fk_director FOREIGN KEY (director_id) REFERENCES
main.dim_director (id)
);

```

-- Movie-Writer Relationship (Many-to-Many)

```

CREATE TABLE main.movie_writer
(

```

```

        movie_id INT NOT NULL,
        writer_id INT NOT NULL,
        PRIMARY KEY (movie_id, writer_id),
        CONSTRAINT fk_movie FOREIGN KEY (movie_id) REFERENCES
main.dim_movie (id),
        CONSTRAINT fk_writer FOREIGN KEY (writer_id) REFERENCES
main.dim_writer (id)
);

```

-- Movie-Star Relationship (Many-to-Many)

```

CREATE TABLE main.movie_star
(
    movie_id INT NOT NULL,
    star_id INT NOT NULL,
    PRIMARY KEY (movie_id, star_id),
    CONSTRAINT fk_movie FOREIGN KEY (movie_id) REFERENCES
main.dim_movie (id),
    CONSTRAINT fk_star FOREIGN KEY (star_id) REFERENCES
main.dim_star (id)
);

```

Код завантаження даних зі stage зони в main:

```
-- Insert data into dim_company
INSERT INTO main.dim_company (name)
SELECT DISTINCT company
FROM staging.movies
WHERE company IS NOT NULL
ON CONFLICT (name) DO NOTHING;

-- Insert data into dim_person
INSERT INTO main.dim_person (name, birth_year, death_year)
SELECT DISTINCT director_name, director_birth_year,
director_death_year
FROM staging.movie_budget
WHERE director_name IS NOT NULL
ON CONFLICT (name) DO NOTHING;

-- Insert data into dim_genre
INSERT INTO main.dim_genre (name)
SELECT DISTINCT(genre)
FROM staging.movies
ON CONFLICT (name) DO NOTHING;

-- Insert data into dim_time
INSERT INTO main.dim_time (year, month, day, quarter, weekday,
iso_week)
SELECT DISTINCT EXTRACT(YEAR FROM released),
EXTRACT(MONTH FROM released),
EXTRACT(DAY FROM released),
EXTRACT(QUARTER FROM released),
TRIM(TO_CHAR(released, 'Day')),
EXTRACT(WEEK FROM released)
FROM staging.movies
ON CONFLICT (year, month, day) DO NOTHING;

-- Insert data into dim_movie
INSERT INTO main.dim_movie (name, release_date, runtime_minutes,
company_id)
SELECT DISTINCT m.name,
m.released,
m.runtime,
c.id AS company_id
FROM staging.movies m
LEFT JOIN main.dim_company c ON m.company = c.name
ON CONFLICT (name) DO NOTHING;

-- Insert data into dim_director (SCD Type 2)
INSERT INTO main.dim_director (person_id, valid_from)
SELECT p.id, NOW()
FROM staging.movie_budget mb
JOIN main.dim_person p ON mb.director_name = p.name
```

```

WHERE NOT EXISTS (
    SELECT 1 FROM main.dim_director d WHERE d.person_id = p.id AND
    d.is_current = TRUE
);

```

```

-- Insert data into fact_movie_performance
INSERT INTO main.fact_movie_performance (movie_id,
production_budget, domestic_gross, worldwide_gross,
movie_average_rating,
movie_number_of_votes, approval_index, time_id)
SELECT dm.id,
    mb.production_budget,
    mb.domestic_gross,
    mb.worldwide_gross,
    mb.movie_average_rating,
    mb.movie_number_of_votes,
    mb.approval_index,
    dt.id AS time_id
FROM staging.movie_budget mb
    JOIN main.dim_movie dm ON mb.movie_title = dm.name
    JOIN main.dim_time dt ON EXTRACT(YEAR FROM
mb.production_date) = dt.year
ON CONFLICT (movie_id, time_id) DO NOTHING;

```

```

-- Insert data into fact_oscar_awards
INSERT INTO main.fact_oscar_awards (movie_id, person_id, category,
canon_category, winner, time_id)
SELECT dm.id, dp.id, oa.category, oa.canon_category, oa.winner,
dt.id
FROM staging.oscar_awards oa
    LEFT JOIN main.dim_movie dm ON oa.film = dm.name
    LEFT JOIN main.dim_person dp ON oa.name = dp.name
    JOIN main.dim_time dt ON oa.year_film = dt.year
WHERE dm.id IS NOT NULL -- Filter out rows with no matching
movie_id
ON CONFLICT (movie_id, time_id, category) DO NOTHING;

```

```

-- Insert movie-genre relationships
INSERT INTO main.movie_genre (movie_id, genre_id)
SELECT DISTINCT dm.id, dg.id
FROM staging.movie_budget mb
    JOIN main.dim_movie dm ON mb.movie_title = dm.name
    JOIN main.dim_genre dg ON dg.name = ANY
(string_to_array(mb.genres, ','))
ON CONFLICT (movie_id, genre_id) DO NOTHING;

```

```

-- Insert movie-director relationships
INSERT INTO main.movie_director (movie_id, director_id)
SELECT DISTINCT dm.id, dd.id
FROM staging.movie_budget mb
    JOIN main.dim_movie dm ON mb.movie_title = dm.name
    JOIN main.dim_person dp ON mb.director_name = dp.name

```

```

        JOIN main.dim_director dd ON dp.id = dd.person_id
ON CONFLICT (movie_id, director_id) DO NOTHING;

-- Insert data into dim_star
INSERT INTO main.dim_star (person_id, valid_from)
SELECT p.id, NOW()
FROM staging.movies m
        JOIN main.dim_person p ON m.star = p.name
WHERE m.star IS NOT NULL
        AND NOT EXISTS (
            SELECT 1 FROM main.dim_star ds WHERE ds.person_id = p.id
AND ds.is_current = TRUE
        );

-- Insert data into dim_writer
INSERT INTO main.dim_writer (person_id, valid_from)
SELECT p.id, NOW()
FROM staging.movies m
        JOIN main.dim_person p ON m.writer = p.name
WHERE m.writer IS NOT NULL
        AND NOT EXISTS (
            SELECT 1 FROM main.dim_writer dw WHERE dw.person_id =
p.id AND dw.is_current = TRUE
        );

-- Insert data into movie_star relationship
INSERT INTO main.movie_star (movie_id, star_id)
SELECT DISTINCT dm.id, ds.id
FROM staging.movies m
        JOIN main.dim_movie dm ON m.name = dm.name
        JOIN main.dim_star ds
            ON ds.person_id = (SELECT id FROM main.dim_person
WHERE name = m.star LIMIT 1)
ON CONFLICT (movie_id, star_id) DO NOTHING;

-- Insert data into movie_writer relationship
INSERT INTO main.movie_writer (movie_id, writer_id)
SELECT DISTINCT dm.id, dw.id
FROM staging.movies m
        JOIN main.dim_movie dm ON m.name = dm.name
        JOIN main.dim_writer dw
            ON dw.person_id = (SELECT id FROM main.dim_person
WHERE name = m.writer LIMIT 1)
ON CONFLICT (movie_id, writer_id) DO NOTHING;

```

dim_director [data_analysis@docker]

WHERE

ORDER BY

	id	person_id	valid_from	valid_to	is_current
1	1	1100	2025-03-23 22:10:14.169023	<null>	• true
2	2	1567	2025-03-23 22:10:14.169023	<null>	• true
3	3	223	2025-03-23 22:10:14.169023	<null>	• true
4	4	1351	2025-03-23 22:10:14.169023	<null>	• true
5	5	1274	2025-03-23 22:10:14.169023	<null>	• true
6	6	1719	2025-03-23 22:10:14.169023	<null>	• true
7	7	97	2025-03-23 22:10:14.169023	<null>	• true
8	8	1351	2025-03-23 22:10:14.169023	<null>	• true
9	9	1141	2025-03-23 22:10:14.169023	<null>	• true
10	10	1728	2025-03-23 22:10:14.169023	<null>	• true
11	11	906	2025-03-23 22:10:14.169023	<null>	• true
12	12	471	2025-03-23 22:10:14.169023	<null>	• true
13	13	1819	2025-03-23 22:10:14.169023	<null>	• true
14	14	1819	2025-03-23 22:10:14.169023	<null>	• true
15	15	1797	2025-03-23 22:10:14.169023	<null>	• true
16	16	1664	2025-03-23 22:10:14.169023	<null>	• true
17	17	1100	2025-03-23 22:10:14.169023	<null>	• true
18	18	783	2025-03-23 22:10:14.169023	<null>	• true
19	19	1408	2025-03-23 22:10:14.169023	<null>	• true
20	20	1034	2025-03-23 22:10:14.169023	<null>	• true
21	21	223	2025-03-23 22:10:14.169023	<null>	• true
22	22	1351	2025-03-23 22:10:14.169023	<null>	• true
23	23	1312	2025-03-23 22:10:14.169023	<null>	• true
24	24	318	2025-03-23 22:10:14.169023	<null>	• true
25	25	1492	2025-03-23 22:10:14.169023	<null>	• true
26	26	1750	2025-03-23 22:10:14.169023	<null>	• true
27	27	1397	2025-03-23 22:10:14.169023	<null>	• true

Рис 5.1 – Заповнена таблиця main зони “dim_director”

dim_movie [data_analysis@docker]

dim_director [data_analysis@docker] dim_movie [data_analysis@docker] ×

1-500 of 501+ | Tx: Auto | DDL | CSV | Download | Refresh | Filter

	1	2	3	4	5
id	1	2	3	4	5
name	Birth	Gosford Park	Moana	Face/Off	The Swee
previous_release_date	<null>	<null>	<null>	<null>	<null>
release_date	2004-10-29	2002-01-18	2016-11-23	1997-06-27	1997-11-
runtime_minutes	100.0	131.0	107.0	138.0	112.0
company_id	2153	561	180	2346	2230

SUM: Not enough values

Рис 5.2 – Заповнена таблиця main зони “dim_movie”

dim_company [data_analysis@docker]

analysis@docker] dim_movie [data_analysis@docker] dim_company [data_analysis@docker] ×

1-500 of 501+ | Tx: Auto | DDL | CSV | Download | Refresh | Filter

	1	2	3	4
id	1	2	3	4
previous_name	<null>	<null>	<null>	<null>
name	Hartbeat Productions	Hammerhead Productions	Electric Panda Entertainment	Mobra Films

SUM: Not enough values

Рис 5.3 – Заповнена таблиця main зони “dim_company”

dim_star [data_analysis@docker]

analysis@docker] dim_movie [data_analysis@docker] dim_star [data_analysis@docker] x

1-500 of 501+ Tx: Auto DDL CSV

WHERE ORDER BY

	id	person_id	valid_from	valid_to	is_current
1	1	19	2025-03-23 22:10:19.559299	<null>	• true
2	2	371	2025-03-23 22:10:19.559299	<null>	• true
3	3	1186	2025-03-23 22:10:19.559299	<null>	• true
4	4	314	2025-03-23 22:10:19.559299	<null>	• true
5	5	371	2025-03-23 22:10:19.559299	<null>	• true
6	6	875	2025-03-23 22:10:19.559299	<null>	• true
7	7	475	2025-03-23 22:10:19.559299	<null>	• true
8	8	715	2025-03-23 22:10:19.559299	<null>	• true
9	9	515	2025-03-23 22:10:19.559299	<null>	• true
10	10	40	2025-03-23 22:10:19.559299	<null>	• true
11	11	976	2025-03-23 22:10:19.559299	<null>	• true
12	12	715	2025-03-23 22:10:19.559299	<null>	• true
13	13	19	2025-03-23 22:10:19.559299	<null>	• true
14	14	1756	2025-03-23 22:10:19.559299	<null>	• true
15	15	95	2025-03-23 22:10:19.559299	<null>	• true
16	16	529	2025-03-23 22:10:19.559299	<null>	• true
17	17	1108	2025-03-23 22:10:19.559299	<null>	• true
18	18	120	2025-03-23 22:10:19.559299	<null>	• true

Рис 5.4 – Заповнена таблиця main зони “dim_star”

dim_writer [data_analysis@docker]

analysis@docker] dim_movie [data_analysis@docker] dim_writer [data_analysis@docker] x

1-500 of 501+ Tx: Auto DDL CSV

WHERE ORDER BY

	id	person_id	valid_from	valid_to	is_current
1	1	901	2025-03-23 22:10:19.581096	<null>	• true
2	2	1614	2025-03-23 22:10:19.581096	<null>	• true
3	3	1760	2025-03-23 22:10:19.581096	<null>	• true
4	4	1396	2025-03-23 22:10:19.581096	<null>	• true
5	5	1829	2025-03-23 22:10:19.581096	<null>	• true
6	6	762	2025-03-23 22:10:19.581096	<null>	• true
7	7	818	2025-03-23 22:10:19.581096	<null>	• true
8	8	711	2025-03-23 22:10:19.581096	<null>	• true
9	9	976	2025-03-23 22:10:19.581096	<null>	• true
10	10	314	2025-03-23 22:10:19.581096	<null>	• true
11	11	1732	2025-03-23 22:10:19.581096	<null>	• true
12	12	1031	2025-03-23 22:10:19.581096	<null>	• true
13	13	1031	2025-03-23 22:10:19.581096	<null>	• true
14	14	1177	2025-03-23 22:10:19.581096	<null>	• true
15	15	1614	2025-03-23 22:10:19.581096	<null>	• true
16	16	1728	2025-03-23 22:10:19.581096	<null>	• true
17	17	420	2025-03-23 22:10:19.581096	<null>	• true
18	18	901	2025-03-23 22:10:19.581096	<null>	• true

Рис 5.5 – Заповнена таблиця main зони “dim_writer”

The screenshot shows a database application window titled 'dim_genre [data_analysis@docker]'. The window displays a table with 19 rows and 2 columns: 'id' and 'name'. The table is sorted by 'id' in ascending order. The genres listed are: Romance, Sport, Mystery, History, Musical, Music, Fantasy, Horror, Drama, Biography, Action, Thriller, Western, Sci-Fi, Comedy, Adventure, Family, Animation, and Crime.

id	name
1	Romance
2	Sport
3	Mystery
4	History
5	Musical
6	Music
7	Fantasy
8	Horror
9	Drama
10	Biography
11	Action
12	Thriller
13	Western
14	Sci-Fi
15	Comedy
16	Adventure
17	Family
18	Animation
19	Crime

Рис 5.6 – Заповнена таблиця main зони “dim_genre”

	1	2	3	4	5	6	7	8	9	10	11
fact_id	1	2	3	4	5	6	7	8	9	10	11
movie_id	4	4	6	9	9	10	10	13	13	16	16
director_id	3260	1426	<null>	2929	1095	1848	14	3483	1649	2727	893
average_rating	7.5	7.5	6.5	7.2	7.2	5.2	5.2	7.0	7.0	5.3	5.3
number_of_votes	194048	194048	336694	198381	198381	14584	14584	88925	88925	2843	2843
approval_index	6.59	6.59	5.95	6.33	6.33	3.54	3.54	5.74	5.74	2.97	2.97
production_budget	17000000.00	17000000.00	44000000.00	6500000.00	6500000.00	3500000.00	3500000.00	10000000.00	10000000.00	20000000.00	20000000.00
domestic_gross	71502303.00	71502303.00	6105175.00	4002293.00	4002293.00	3293258.00	3293258.00	25697647.00	25697647.00	18826490.00	18826490.00
worldwide_gross	109726800.00	109726800.00	12342632.00	20412841.00	20412841.00	3293258.00	3293258.00	25707698.00	25707698.00	18826490.00	18826490.00

Рис 5.7 – Заповнена таблиця main зони “fact_movie_performance”

The screenshot shows a web-based data analysis interface with three tabs: 'analysis@docker', 'dim_movie [data_analysis@docker]', and 'dim_person [data_analysis@docker]'. The 'dim_person' tab is active, displaying a table with 20 rows. The table has four columns: 'id', 'name', 'birth_year', and 'death_year'. The data is sorted by 'id' in ascending order. The 'death_year' column contains null values for several entries.

id	name	birth_year	death_year
1	Curtis Hanson	1945	2016
2	Bruce Beresford	1940	<null>
3	Ol Parker	1969	<null>
4	Rob Letterman	<null>	<null>
5	Bob Saget	1956	2022
6	Bryan Bertino	1977	<null>
7	Diane Keaton	1946	<null>
8	David M. Rosenthal	1969	<null>
9	Frank LaLoggia	1954	<null>
10	Gregory Jacobs	<null>	<null>
11	Steven S. DeKnight	<null>	<null>
12	John Carl Buechler	1952	2019
13	Dibakar Banerjee	1969	<null>
14	John Moore	1970	<null>
15	Atom Egoyan	1960	<null>
16	Terry Zwigoff	1949	<null>
17	Steve Rash	<null>	<null>
18	Vincent Gallo	1961	<null>
19	Robert De Niro	1943	<null>
20	Tom Vaughan	1969	<null>

Рис 5.8 – Заповнена таблиця main зони “dim_person”

dim_time [data_analysis@docker]

analysis@docker] dim_movie [data_analysis@docker] dim_time [data_analysis@docker] x

1-500 of 501+ Tx: Auto DDL CSV

WHERE ORDER BY

	id	year	month	day	quarter	weekday	iso_week
1	1	1993	9	8	3	Wednesday	36
2	2	1991	5	3	2	Friday	18
3	3	1992	7	10	3	Friday	28
4	4	2010	3	14	1	Sunday	10
5	5	1999	8	4	3	Wednesday	31
6	6	2001	5	4	2	Friday	18
7	7	2001	3	7	1	Wednesday	10
8	8	1994	4	15	2	Friday	15
9	9	2017	3	31	1	Friday	13
10	10	1998	1	23	1	Friday	4
11	11	1990	3	2	1	Friday	9
12	12	2015	12	25	4	Friday	52
13	13	1986	9	12	3	Friday	37
14	14	1998	6	12	2	Friday	24
15	15	1987	10	2	4	Friday	40
16	16	2011	1	20	1	Thursday	3
17	17	1999	6	4	2	Friday	22
18	18	1982	10	29	4	Friday	43
19	19	2018	7	20	3	Friday	29
20	20	1990	9	7	3	Friday	36
21	21	1984	9	19	3	Wednesday	38
22	22	2005	12	23	4	Friday	51

Рис 5.9 – Заповнена таблиця main зони “dim_time”

fact_movie_performance [data_analysis@docker]

1-500 of 501+

WHERE ORDER BY

	1	2	3	4	5	6	7
id	1	2	3	4	5	6	7
movie_id	6313	4220	2751	7371	3570	6	68
production_budget	22000000.00	4500000.00	350000.00	15000000.00	550000.00	1100000.00	40
domestic_gross	15716828.00	47923795.00	10000000.00	69847348.00	39754601.00	21445318.00	70
worldwide_gross	15716828.00	47923795.00	10000000.00	69847348.00	59754601.00	21445868.00	70
domestic_percent	100.00	100.00	100.00	100.00	66.53	100.00	10
foreign_gross	0.00	0.00	0.00	0.00	20000000.00	550.00	0.
foreign_percent	0.00	0.00	0.00	0.00	33.47	0.00	0.
movie_average_rating	5.3	5.8	6.4	6.2	6.4	6.8	5.
movie_number_of_votes	4482	72178	18516	27095	143679	76567	48
approval_index	3.15	4.64	4.50	4.52	5.46	5.49	3.
time_id	97	97	97	97	97	97	97

SUM: Not enough values

Рис 5.10 – Заповнена таблиця main зони “fact_movie_performance”

fact_oscar_awards [data_analysis@docker]

1-500 of 501+

WHERE ORDER BY

	1	2	3
id	1	2	3
movie_id	5924	5924	5924
person_id	<null>	<null>	<null>
category	WRITING (Adapted Screenplay)	WRITING (Adapted Screenplay)	WRITING (Adapted Screenpla
canon_category	WRITING (Adapted Screenplay)	WRITING (Adapted Screenplay)	WRITING (Adapted Screenpla
winner	true •	true •	true •
time_id	865	1838	2520

SUM: Not enough values

Рис 5.11 – Заповнена таблиця main зони “fact_oscar_awards”

Завантаження змінених даних

База даних містить інформацію про фільми, режисерів, сценаристів, акторів, нагороди та касові збори. Деякі з цих даних можуть змінюватися з часом, тому застосовується SCD (Slowly Changing Dimensions) для їхнього правильного зберігання та обробки.

Таблиця `dim_company` – SCD Type 3 (збереження попереднього значення)

Ця таблиця містить інформацію про кінокомпанії, які беруть участь у створенні фільмів. Застосовується **SCD Type 3**, що дозволяє зберігати як актуальну, так і попередню назву компанії.

- Якщо компанія змінює назву, її попереднє значення зберігається у стовпці `previous_name`, а нове записується у `name`.
- Таким чином, можна аналізувати зміни в назвах компаній, не дублюючи записи.

Таблиця `dim_person` – SCD Type 1 (перезапис значень)

Ця таблиця містить інформацію про персоналії, такі як актори, режисери та сценаристи. Застосовується **SCD Type 1**, оскільки оновлення даних (наприклад, виправлення помилок, оновлення року смерті) не потребує збереження історії змін.

- Якщо змінюється будь-який атрибут (ім'я, рік народження або смерті), запис оновлюється без створення нової версії.

Таблиця `dim_genre` – SCD Type 1 (перезапис значень)

Ця таблиця містить інформацію про жанри фільмів. Застосовується **SCD Type 1**, оскільки історія змін назв жанрів не є критично важливою.

- Якщо жанр перейменовується або уточнюється, його значення просто оновлюється без збереження попередніх версій.

Таблиця `dim_movie` – SCD Type 3 (збереження попереднього значення)

Ця таблиця містить основну інформацію про фільми, включаючи назву, дату випуску, тривалість та компанію-виробника. Застосовується **SCD Type 3**, що дозволяє зберігати попередню дату випуску:

- Якщо змінюється дата випуску фільму, її старе значення переноситься у `previous_release_date`, а нове записується у `release_date`.
- Це дозволяє аналізувати вплив змін у даті випуску на успіх фільму.

Таблиця `dim_director` – SCD Type 2 (збереження історії змін)

Ця таблиця містить інформацію про режисерів фільмів. Застосовується **SCD Type 2**, оскільки режисерський склад фільму може змінюватися з часом.

- При зміні режисера створюється новий запис, а попередній позначається як неактуальний (`valid_to` отримує значення дати завершення періоду дії, а `is_current` змінюється на `FALSE`).
- Це дозволяє аналізувати історичні зміни у складі режисерів фільмів.

Таблиця `dim_writer` – SCD Type 2 (збереження історії змін)

Ця таблиця містить інформацію про сценаристів фільмів. Застосовується **SCD Type 2**, оскільки список сценаристів фільму може змінюватися з часом.

- Якщо сценарист змінюється, створюється новий запис із новими значеннями, а старий запис позначається як неактуальний.
- Це дозволяє зберігати повну історію змін сценаристів.

Таблиця `dim_star` – SCD Type 2 (збереження історії змін)

Ця таблиця містить інформацію про акторів, які знімалися у фільмах. Застосовується **SCD Type 2**, оскільки акторський склад може змінюватися.

- Якщо змінюється інформація про участь актора у фільмі, створюється новий запис, а старий позначається як неактуальний.
- Це дозволяє зберігати історію змін акторського складу.

Код:

```
-- SCD type 3 for dim_movie
CREATE OR REPLACE FUNCTION main.update_dim_movie(
    p_id INT,
    p_new_name TEXT,
    p_new_release_date DATE
)
RETURNS VOID AS $$
BEGIN
    UPDATE main.dim_movie
```

```

        SET previous_release_date = release_date, -- Store old value
            release_date = p_new_release_date, -- Overwrite with new
value
            name = p_new_name
        WHERE id = p_id;
END;
$$ LANGUAGE plpgsql;

-- SCD type 3 for dim_company
CREATE OR REPLACE FUNCTION main.update_dim_company(
    p_id INT,
    p_new_name TEXT
)
RETURNS VOID AS $$
BEGIN
    UPDATE main.dim_company
    SET previous_name = name, -- Store old value
        name = p_new_name -- Overwrite with new value
    WHERE id = p_id;
END;
$$ LANGUAGE plpgsql;

-- SCD type 2 for dim_director
CREATE OR REPLACE FUNCTION main.update_dim_director(
    p_person_id INT
)
RETURNS VOID AS $$
BEGIN
    -- Set the current record to expired
    UPDATE main.dim_director
    SET valid_to = now(), is_current = FALSE
    WHERE person_id = p_person_id AND is_current = TRUE;

    -- Insert a new record
    INSERT INTO main.dim_director (person_id, valid_from, valid_to,
is_current)
    VALUES (p_person_id, now(), NULL, TRUE);
END;
$$ LANGUAGE plpgsql;

-- SCD type 2 for dim_writer
CREATE OR REPLACE FUNCTION main.update_dim_writer(
    p_person_id INT
)
RETURNS VOID AS $$
BEGIN
    UPDATE main.dim_writer
    SET valid_to = now(), is_current = FALSE
    WHERE person_id = p_person_id AND is_current = TRUE;

    INSERT INTO main.dim_writer (person_id, valid_from, valid_to,
is_current)

```

```

VALUES (p_person_id, now(), NULL, TRUE);
END;
$$ LANGUAGE plpgsql;

-- SCD type 2 for dim_star
CREATE OR REPLACE FUNCTION main.update_dim_star(
    p_person_id INT
)
RETURNS VOID AS $$
BEGIN
    UPDATE main.dim_star
    SET valid_to = now(), is_current = FALSE
    WHERE person_id = p_person_id AND is_current = TRUE;

    INSERT INTO main.dim_star (person_id, valid_from, valid_to,
is_current)
VALUES (p_person_id, now(), NULL, TRUE);
END;
$$ LANGUAGE plpgsql;

```

Завантаження нових даних

База даних містить фактологічні таблиці (fact_box_office, fact_movie_performance, fact_oscar_awards), а також зв'язкові таблиці (movie_stars, movie_writers, movie_genre, movie_director). Оскільки ці таблиці містять динамічні дані, використовується інкрементальне завантаження для оновлення та додавання нових записів без дублювання.

1. fact_box_office – оновлення фінансових показників

Опис:

Містить інформацію про касові збори фільмів, які можуть змінюватися з часом.

Метод оновлення:

- Якщо запис вже існує (визначається за movie_id та time_id), оновлюються дані про світові та локальні збори.
- Якщо запису немає, він додається як новий.

Це дозволяє коригувати попередні фінансові дані, які можуть уточнюватися після виходу фільму.

2. fact_movie_performance – оновлення рейтингів, зборів та бюджету

Опис:

Містить оцінки фільмів, кількість голосів та бюджетні показники.

Метод оновлення:

- Якщо запис вже існує (movie_id, time_id), оновлюються:
 - середній рейтинг
 - кількість голосів
 - касові збори
 - бюджет
- Якщо запису немає, додається новий.

Такий підхід важливий, оскільки рейтинг фільму та кількість голосів змінюються з часом (наприклад, через нові оцінки користувачів).

3. **fact_oscar_awards** – додавання нових номінацій та переможців

Опис:

Містить інформацію про номінації та перемоги на «Оскарі».

Метод оновлення:

- Якщо фільм уже був номінований у категорії (movie_id, category, time_id), запис не змінюється (щоб зберегти історичні дані).
- Якщо з'являється нова номінація або переможець, запис додається.

Це дозволяє правильно відстежувати оновлення списку номінантів та переможців.

4. **movie_stars** та **movie_writers** – унікальні зв'язки

Опис:

Ці таблиці містять зв'язки між фільмами та акторами (movie_stars) і сценаристами (movie_writers).

Метод оновлення:

- Якщо зв'язок вже існує, він не дублюється.
- Якщо актор або сценарист новий для фільму, запис додається.

Це забезпечує коректне відображення акторського складу та команди фільму без зайвих записів.

5. **movie_genre** – оновлення жанрів фільму

Опис:

Зв'язкова таблиця, яка містить відношення між фільмами та їх жанрами.

Метод оновлення:

- Якщо жанр вже присвоєний фільму, новий запис не створюється.
- Якщо фільм має новий жанр, він додається в таблицю.

Це дозволяє відстежувати жанрову приналежність фільмів без дублювання записів.

6. **movie_director** – оновлення інформації про режисерів

Опис:

Містить зв'язки між фільмами та їхніми режисерами.

Метод оновлення:

- Якщо зв'язок між фільмом і режисером уже існує, він не дублюється.
- Якщо у фільму з'явився новий режисер, новий запис додається.

Це забезпечує актуальність інформації про режисерів без створення зайвих записів.

Код:

```
INSERT INTO main.dim_company (name, previous_name)
SELECT DISTINCT company, NULL
FROM staging.movies s
WHERE company IS NOT NULL
  AND NOT EXISTS (SELECT 1
                  FROM main.dim_company c
                  WHERE c.name = s.company)
ON CONFLICT (name) DO NOTHING;
```

```
INSERT INTO main.dim_person (name, birth_year, death_year)
SELECT DISTINCT director_name, director_birth_year,
director_death_year
FROM staging.movie_budget s
WHERE director_name IS NOT NULL
  AND NOT EXISTS (SELECT 1
                  FROM main.dim_person p
                  WHERE p.name = s.director_name
                     AND p.birth_year = s.director_birth_year
                     AND p.death_year = s.director_death_year)
ON CONFLICT (name, birth_year, death_year) DO NOTHING;
```

```
INSERT INTO main.dim_genre (name)
SELECT DISTINCT genre
FROM staging.movies s
WHERE NOT EXISTS (SELECT 1
                  FROM main.dim_genre g
                  WHERE g.name = s.genre)
ON CONFLICT (name) DO NOTHING;
```

```

INSERT INTO main.dim_time (year, month, day, quarter, weekday,
iso_week)
SELECT DISTINCT EXTRACT(YEAR FROM released),
                EXTRACT(MONTH FROM released),
                EXTRACT(DAY FROM released),
                EXTRACT(QUARTER FROM released),
                TRIM(TO_CHAR(released, 'Day')),
                EXTRACT(WEEK FROM released)
FROM staging.movies s
WHERE NOT EXISTS (SELECT 1
                  FROM main.dim_time t
                  WHERE t.year = EXTRACT(YEAR FROM s.released)
                     AND t.month = EXTRACT(MONTH FROM s.released)
                     AND t.day = EXTRACT(DAY FROM s.released))
ON CONFLICT (year, month, day) DO NOTHING;

INSERT INTO main.dim_movie (name, release_date,
previous_release_date, runtime_minutes, company_id)
SELECT DISTINCT s.name,
                s.released,
                NULL,
                s.runtime,
                c.id AS company_id
FROM staging.movies s
     LEFT JOIN main.dim_company c ON s.company = c.name
WHERE NOT EXISTS (SELECT 1
                  FROM main.dim_movie m
                  WHERE m.name = s.name)
ON CONFLICT (name)
DO UPDATE SET previous_release_date =
main.dim_movie.release_date,
               release_date          = EXCLUDED.release_date;

WITH new_directors AS (SELECT DISTINCT p.id AS person_id
                       FROM staging.movie_budget s
                       JOIN main.dim_person p ON
s.director_name = p.name
                       WHERE NOT EXISTS (SELECT 1
                                         FROM main.dim_director d
                                         WHERE d.person_id = p.id
                                         AND d.is_current =
TRUE))
UPDATE main.dim_director
SET valid_to   = NOW(),
    is_current = FALSE
WHERE person_id IN (SELECT person_id FROM new_directors)
    AND is_current = TRUE;

WITH new_directors AS (SELECT DISTINCT p.id AS person_id

```

```

FROM staging.movie_budget s
      JOIN main.dim_person p ON
s.director_name = p.name
      WHERE NOT EXISTS (SELECT 1
                        FROM main.dim_director d
                        WHERE d.person_id = p.id
                        AND d.is_current =
TRUE))
INSERT
INTO main.dim_director (person_id, valid_from)
SELECT person_id, NOW()
FROM new_directors;

```

```

INSERT INTO main.fact_movie_performance (movie_id,
production_budget, domestic_gross, worldwide_gross,
movie_average_rating,
movie_number_of_votes, approval_index, time_id)
SELECT dm.id,
s.production_budget,
s.domestic_gross,
s.worldwide_gross,
s.movie_average_rating,
s.movie_number_of_votes,
s.approval_index,
dt.id AS time_id
FROM staging.movie_budget s
      JOIN main.dim_movie dm ON s.movie_title = dm.name
      JOIN main.dim_time dt ON EXTRACT(YEAR FROM
s.production_date) = dt.year
ON CONFLICT (movie_id, time_id) DO UPDATE
SET production_budget = EXCLUDED.production_budget,
domestic_gross = EXCLUDED.domestic_gross,
worldwide_gross = EXCLUDED.worldwide_gross,
movie_average_rating = EXCLUDED.movie_average_rating,
movie_number_of_votes = EXCLUDED.movie_number_of_votes,
approval_index = EXCLUDED.approval_index;

```

```

INSERT INTO main.fact_oscar_awards (movie_id, person_id, category,
canon_category, winner, time_id)
SELECT dm.id, dp.id, s.category, s.canon_category, s.winner, dt.id
FROM staging.oscar_awards s
      LEFT JOIN main.dim_movie dm ON s.film = dm.name
      LEFT JOIN main.dim_person dp ON s.name = dp.name
      JOIN main.dim_time dt ON s.year_film = dt.year
WHERE dm.id IS NOT NULL
ON CONFLICT (movie_id, time_id, category) DO NOTHING;

```

```

INSERT INTO main.movie_genre (movie_id, genre_id)
SELECT DISTINCT dm.id, dg.id
FROM staging.movie_budget s

```


[illegible]

```

UPDATE main.dim_writer
SET valid_to = NOW(),
    is_current = FALSE
WHERE person_id IN (SELECT person_id FROM new_writers)
    AND is_current = TRUE;

WITH new_writers AS (SELECT DISTINCT p.id AS person_id
                     FROM staging.movies s
                     JOIN main.dim_person p ON s.writer =
p.name
                     WHERE NOT EXISTS (SELECT 1
                                       FROM main.dim_writer dw
                                       WHERE dw.person_id = p.id
                                       AND dw.is_current = TRUE))

INSERT
INTO main.dim_writer (person_id, valid_from)
SELECT person_id, NOW()
FROM new_writers;

INSERT INTO main.movie_star (movie_id, star_id)
SELECT DISTINCT dm.id, ds.id
FROM staging.movies s
    JOIN main.dim_movie dm ON s.name = dm.name
    JOIN main.dim_star ds ON ds.person_id = (SELECT id FROM
main.dim_person WHERE name = s.star LIMIT 1)
ON CONFLICT (movie_id, star_id) DO NOTHING;

INSERT INTO main.movie_writer (movie_id, writer_id)
SELECT DISTINCT dm.id, dw.id
FROM staging.movies s
    JOIN main.dim_movie dm ON s.name = dm.name
    JOIN main.dim_writer dw ON dw.person_id = (SELECT id FROM
main.dim_person WHERE name = s.writer LIMIT 1)
ON CONFLICT (movie_id, writer_id) DO NOTHING;

```

Висновок:

У ході виконання лабораторної роботи було здійснено збір та підготовку даних, що включало вибір кількох джерел відкритої інформації, пов'язаної з індустрією кіно, а також очищення та трансформацію цих даних для подальшого використання. Було спроектовано модель сховища даних, створено таблиці для Stage зони та основного сховища, а також розроблено ETL-процеси, які забезпечують завантаження даних на проміжний етап та їх подальше перетворення для основного сховища. Реалізовано процеси завантаження та оновлення даних, зокрема методи SCD для обробки змінних даних, що дозволило підтримувати історичну

точність інформації. В результаті дані були успішно підготовлені для подальшого аналізу, а створені таблиці вимірів і фактів дозволяють виконувати широкий спектр аналітичних запитів. Таким чином, усі поставлені завдання виконано, що дозволило створити ефективну систему для зберігання та аналізу даних про кіноіндустрію.